QATRA — Web App Build Plan

Task Distribution & Team Execution Guide

Repository: https://github.com/abdulhayykhan/QATRA-Web-App/
Stack: FastAPI (backend) · Vanilla HTML/CSS/JS (frontend) · Deployment on Vercel
Prepared by: Abdul Hayy Khan — Team Lead
Based on: QATRA PRD v1.0 & Wireframes (Approved by Team)

A note before we start: this document is the single source of truth for who is building what, in what order. Our PRD and wireframes were written for a native mobile app — we are now building this as a responsive web application, so every screen in the wireframe PDF needs to be reinterpreted for the browser before it's coded. Please read this whole document once before opening the repo. If something here contradicts the PRD, this document wins, since it reflects our actual tech decisions.

Two decisions that override the PRD:
1) Database is **Supabase** (managed Postgres) — confirmed, not open for debate.
2) Identity is **Firebase Authentication using "Continue with Google"** — we are dropping phone number + OTP verification entirely because we can't afford an SMS gateway right now. Wherever the PRD or wireframes mention Phone OTP, read it as Google Sign-In instead. CNIC upload/verification stays, it just now attaches to a Google-authenticated account instead of a phone-verified one.

1. Repository & Workflow Rules

All work happens in **https://github.com/abdulhayykhan/QATRA-Web-App/**. No exceptions, no personal forks used as the "real" version — if it's not on this repo, it doesn't exist for the project.

Branching & PR rules

- **main** is protected. Nobody pushes to main directly, including me.
- Create a branch per task: **feature/<module>-<short-task>** — e.g. *feature/auth-google-signin*, *feature/map-proximity-query*, *feature/feed-frontend*.
- Every branch becomes a Pull Request into **develop** (we merge develop → main only at the end of each phase, after testing).
- **GitHub Copilot is enabled as a required reviewer on every PR.** Do not merge until Copilot's review comments are either resolved or explicitly replied to with a reason you're not changing it.
- In addition to Copilot, one teammate (not the author) must approve every PR before merge.
- PR description must state: which PRD section this implements, which files/endpoints it touches, and how you tested it.
- Keep PRs small and scoped to ONE feature slice. Do not bundle backend + frontend + unrelated fixes in one PR.
- Commit messages: **[module] short description** — e.g. *[map] add haversine proximity query*.

Working with AI tools (Copilot, ChatGPT, Claude, etc.)

We are doing AI-driven development, which is great for speed, but it comes with rules because unsupervised AI code is how projects quietly rot:

- Always give the AI tool the relevant PRD section AND the **api-contract.md** file (see Phase 1) as context — not the whole PRD, and never let it invent its own endpoint names or response formats.
- Read every line the AI generates before committing. You are responsible for code in your PR, not the AI.
- Do not let AI add new pip/npm packages without flagging it in the PR description — Vercel's Python runtime has size limits and a random new dependency can break deployment for everyone.

- Never let AI hardcode API keys, secrets, or CNIC/test data that looks real. Use placeholders and .env files (.env is already in .gitignore — keep it that way).
- If Copilot's PR review flags a security or privacy issue (e.g. phone number exposed, missing auth check) — that is not optional to fix. This app handles CNIC and health data; we treat those flags seriously.

2. Team Roles & Ownership

Each of you owns a full vertical slice of one feature — backend routes AND the frontend pages for that feature. This is deliberate: it's far easier for you (and for Copilot reviewing your PRs) to reason about one person's complete feature than to split every feature across two people's half-finished code.

Member	Owns	PRD Section
Abdul Hayy Khan (Lead)	Architecture, shared DB models/schemas, repo setup, deployment config, final integration	Sec. 1, 2, 8
Hareem Israr	Feature 1 — Live Map & Proximity Matching (backend + frontend)	Sec. 3
Saghir Ahmed	Feature 2 — Auth, Verification, RBAC (backend + frontend)	Sec. 4
Mahrukh Baig	Feature 3 — Social/Urgent Request Feed (backend + frontend)	Sec. 5
Yumna Abbasi	Feature 4 — Awareness & Eligibility Module (backend + frontend)	Sec. 6
Nimra Iftikhar	Non-Functional Requirements — security, rate limiting, logging, testing, CI	Sec. 7

Important: nobody edits another person's router or page files directly. If you need a change in someone else's module, open an issue tagging them or ask in the group — don't silently patch their code in your PR.

---

3. Build Phases — Overview

We work in six phases, in order. Phases 0 and 1 are mostly me + Nimra, so the rest of you aren't blocked waiting on shared foundations. Phases 2 and 3 are where everyone works in parallel on their own module. We move to the next phase once the current one is actually done — not on a fixed date, so don't start frontend work before your backend endpoints exist and match the contract.

Phase	Focus	Who
0	Foundation — repo, Supabase DB, shared schemas, deploy skeleton	Abdul Hayy + Nimra
1	API contract — lock every endpoint before code is written	Abdul Hayy (all review)
2	Backend — each feature's FastAPI router + logic	All 4 feature owners (parallel)
3	Frontend — wireframes → responsive HTML/CSS/JS pages	All 4 feature owners (parallel)
4	Integration — wire features together, RBAC enforcement, journeys	Everyone
5	Testing, security hardening & deployment	Nimra leads, all support

Phase 0 — Foundation

Nobody else starts coding features until Phase 0 is merged to develop. This avoids five people inventing five different User models.

ABDUL HAYY KHAN

- Initialize the repo structure exactly as agreed: /backend/app (routers, models, schemas, services, core), /frontend (pages, static/js, static/css), /docs, /tests.
- Set up FastAPI app skeleton (main.py) with a working health-check endpoint (**GET /api/health**).
- Set up the database on **Supabase** (Postgres) — this is our confirmed choice. Confirm whether PostGIS is enabled on our Supabase project for geo queries; if not, we fall back to plain lat/lng columns with haversine math in SQL, which is fine for our scale.
- Define ALL shared SQLAlchemy models up front in /backend/app/models: User, Request, Donor, Notification, Event, Registration, AuditLog — even with stub fields for things other features will fill in later.
- Define ALL shared Pydantic schemas in /backend/app/schemas to match.
- Set up vercel.json for a FastAPI + static file deployment and confirm the health-check endpoint deploys successfully before anyone builds on top of it.
- Turn on branch protection on main: PR required, Copilot review required, 1 human approval required.
- Create the Firebase project for authentication (Google Sign-In provider only, per our decision to drop phone OTP), and share the frontend config keys + backend service account credentials with the team through .env, never committed to the repo.
- Create develop branch — this is where all feature PRs land first.

NIMRA IFTIKHAR

- Set up the base pytest structure in /tests with a test-database fixture everyone else's tests will use.
- Add rate-limiting middleware skeleton (for login/verification-heavy endpoints per NFR 2.6) so Saghir can plug into it in Phase 2.
- Add an audit-logging helper (per NFR 2.5) that other routers can call when touching CNIC/health data.
- Add a basic encryption helper (AES-256, per NFR 2.1) for sensitive fields — CNIC, health answers.
- Set up GitHub Actions CI: run pytest + a lint check (ruff or flake8) on every PR automatically, in addition to Copilot review.

Nimra's Phase 0 work is not glamorous but it's what stops the rest of us from re-inventing security and logging four different ways. Please build against these helpers instead of writing your own.

Phase 1 — API Contract (Lock Before Coding)

Before any feature router is built, we write down every single endpoint in **/docs/api-contract.md** — method, path, request body, response body. This is the file you and your AI tool point to when generating code. It stops four people from building four different API styles.

ABDUL HAYY KHAN — drafts, everyone reviews

- Draft the contract for all endpoints across Sections 3–6 of the PRD (map, auth, feed, awareness).
- Circulate in the group chat / a dedicated PR for review — every feature owner must comment or approve before we consider it locked.
- Any change to a locked endpoint later requires a PR to api-contract.md first, reviewed by me, before touching code.

Example of the level of detail expected in the contract:

> ***POST /api/requests*** *— create emergency blood request (auth required: verified seeker) → body: {blood_group, component_type, units, hospital_id, urgency} → returns {request_id, status: "pending_verification"}*

Phase 2 — Backend (Parallel, per Feature Owner)

Everyone builds their FastAPI router against the locked contract and shared models from Phase 0. Work in your own router file only. Do not modify shared models/schemas without a separate PR reviewed by me.

HAREEM ISRAR — Feature 1: Live Map & Proximity Matching

- Build /backend/app/routers/map.py: donor location update endpoint (respect the 2–5 min throttle from FR 1.1.2, not continuous streaming).
- Proximity query service (services/geo.py): haversine-based nearest-donor lookup, radius-bounded (5/10/15 km per FR 1.3.2).
- Geo-fenced dispatch logic: filter by blood-group compatibility, cooldown status, pre-screening pass.
- Radius auto-expansion logic (FR 1.3.3) — important: Vercel serverless has no long-running background workers, so this cannot be a sleeping timer. Design it as a re-checkable state (e.g. a scheduled endpoint hit by an external cron service, or checked on next donor poll) — flag this design choice to me before building it.
- Endpoint for request status (searching / donor matched / fulfilled) that the seeker's frontend polls.
- Proximity-ranked donor list output (FR 1.4) sorted by distance, deprioritizing (not excluding) past decliners.

SAGHIR AHMED — Feature 2: Auth, Verification & RBAC

- Build /backend/app/routers/auth.py: verify the Firebase ID token sent from the frontend after "Continue with Google" sign-in, and create/fetch the matching user record in our own DB — this replaces the phone OTP flow in the PRD entirely.
- CNIC checksum validation + front/back image upload endpoint (FR 2.1.2), now attached to the Google-authenticated account instead of a phone-verified one.
- Hospital slip upload + OCR pipeline (FR 2.2) — decide and confirm an OCR approach that fits Vercel's package size limits (pytesseract locally is heavy; consider a hosted OCR API) before building. Confirm with me first.
- Admin escalation queue for low-confidence OCR results (<85%, FR 2.2.3) with approve/reject endpoints.
- RBAC dependency (require_role()) that Map/Feed/Awareness routers will import and apply to their own endpoints.
- Session handling: once Firebase confirms identity, issue our own app-level session/JWT so we're not calling Firebase on every single request.
- 90-day cooldown engine (FR 2.4) and pre-screening health checklist scoring (FR 2.5) as reusable services other routers (Map) will query.
- Use Nimra's encryption helper for all CNIC/health data — do not write your own.

MAHRUKH BAIG — Feature 3: Urgent Request & Awareness Feed

- Build /backend/app/routers/feed.py: structured post creation (FR 3.1) — this reuses the request object from Saghir's verification flow, don't duplicate it.
- Feed filtering endpoint: blood group, urgency, location, drive events (FR 3.2) — query params, not client-side filtering.
- "I Can Donate" one-tap response endpoint + share-link generation (FR 3.3).
- Request lifecycle auto-close logic once required units are fulfilled (FR 3.4).
- Notification trigger logic for rare vs common blood groups (Sec 5.3) — hook into the shared notification service (coordinate with Hareem since Map also dispatches alerts through the same pipeline).

YUMNA ABBASI — Feature 4: Awareness Sessions & Eligibility

- Build /backend/app/routers/awareness.py: 4-step eligibility checker as a stateless scoring endpoint (FR 4.1) — clearly return one of Eligible / May Need Confirmation / Not Eligible, with the disclaimer text from the PRD.
- Awareness content library CRUD (FR 4.2) — articles, videos, FAQs, myth-vs-fact, categorized as in Sec 6.2.
- Event registration & booking endpoints (FR 4.3): browse events → register as donor/volunteer → confirmation.
- Health feedback storage endpoint (FR 4.4) — screening outcome, post-donation instructions, tied to donation record.
- This module feeds donor/event records the Verification and Map modules also read — confirm field names match the shared schema from Phase 0 exactly.

NIMRA IFTIKHAR — supports everyone in Phase 2

- Review every PR for NFR compliance: is CNIC/health data going through the encryption helper? Is the endpoint rate-limited if it's a login/verification endpoint? Is access being logged?
- Start writing integration-style tests as routers land, not after Phase 2 ends.

Phase 3 — Frontend (Wireframes → Responsive Web)

The wireframe PDF shows native mobile screens. Our job is not to copy them pixel-for-pixel — it's to translate the same information and flow into a responsive web page (works on both desktop and mobile browser widths). Keep the QATRA red/white visual identity from the wireframes.

Shared frontend rules — read before writing any HTML/CSS/JS

- No frameworks, no bundlers (no React/Vue/Webpack) — plain HTML, CSS, and JS with **<script type="module">**.
- One JS file per page, same name as the HTML file, in /frontend/static/js.
- ALL API calls go through the shared /frontend/static/js/api.js helper (apiGet/apiPost/etc.) — do not write scattered inline fetch() calls, or error handling will be inconsistent across the app.
- Shared design tokens live in /frontend/static/css/tokens.css (QATRA red, spacing, fonts) — pull your page styles from these variables, don't invent your own palette.
- Every page must handle a failed API call gracefully (show an error state, no blank screen, no console crash).
- Mobile-first CSS: design for a narrow screen first, then expand — most of our real users will be on phones even though this is a web app.

HAREEM ISRAR — Map frontend pages

- Live map view (wireframe pg. 16) — donor's map with radius rings and request summary card.
- Request status / matchmaker dashboard (wireframe pg. 8–9) — seeker's view of donors responding, distance, ETA.
- Masked call/chat coordination screen (wireframe pg. 10, 17) — proxy contact UI.
- Geo-fenced push alert — since this is web, translate the "push notification modal" (wireframe pg. 15) into an in-app banner/toast + browser Notification API where permitted, not a real OS push.

SAGHIR AHMED — Auth & verification frontend pages

- Onboarding/splash screen (wireframe pg. 2) — dual choice: Need Blood Urgently / Register as Donor, leading into "Continue with Google" instead of a phone number screen.
- Google Sign-In flow using Firebase's web SDK (replaces the OTP screen on pg. 3 entirely), Profile setup (pg. 4) — note the CNIC field is no longer optional context, it's part of the required verification step, CNIC upload (pg. 12), Pre-screening checklist (pg. 13).
- Hospital slip upload with camera/file picker (pg. 6) and verification-in-progress state (pg. 7).
- Admin login/2FA (pg. 20), 24/7 verification queue split view (pg. 21), Fraud audit table (pg. 22).

MAHRUKH BAIG — Feed frontend pages

- Feed screen with Urgent/Awareness tabs and filter chips (wireframe pg. 22 shows the reference design language).
- Create Request form (pg. 5) — blood group, component type, units, hospital search, urgency tier.
- Post card component: reusable across feed and matched-donor views — build it once, others will reuse it.
- Donation confirmation & closure screen (pg. 11).

YUMNA ABBASI — Awareness frontend pages

- Donor home dashboard (pg. 14) — Available to Donate toggle, eligibility status, local alerts.
- Eligibility checker — 4-step quiz UI matching FR 4.1 with a clear results screen and disclaimer.
- Awareness content library browse/read view, myth-vs-fact section.
- Event registration flow, cooldown state view (pg. 19), donation complete/cooldown activation screen (pg. 18).

ABDUL HAYY KHAN — cross-cutting frontend

- Build /frontend/static/css/tokens.css and the shared api.js helper before Phase 3 starts, so nobody's blocked.
- Drive Management admin dashboard (wireframe pg. 23) — since this doesn't map cleanly to one feature owner.
- Review all frontend PRs for visual/UX consistency across pages before Phase 4 integration.

Phase 4 — Integration

This is where we stop being four separate features and become one app. Everyone is involved.

- Merge all feature branches into develop; resolve conflicts together, don't silently overwrite someone else's file.
- Walk through all three PRD user journeys end-to-end on the staging deployment: Journey A (Seeker), Journey B (Donor), Journey C (Campus Drive Lead) — I'll lead this with Nimra, everyone else fixes bugs filed against their own module.
- Cross-check RBAC: confirm Saghir's require_role() dependency is actually applied on every Map/Feed/Awareness endpoint that needs it — this is the most common gap when features are built in isolation.
- Confirm phone numbers are never exposed in any API response outside the masked-contact flow — grep every schema for raw phone fields.
- Confirm notification dispatch is shared, not duplicated, between Map's geo-fenced alerts and Feed's rare-blood-group alerts.

Phase 5 — Testing, Hardening & Deployment

NIMRA IFTIKHAR — leads this phase

- Consolidate the pytest suite — each feature owner should have already written tests for their own endpoints as part of their Phase 2 PRs; this phase is about filling gaps and running everything together.
- Security pass against NFR 2.x: rate limiting on login/verification endpoints confirmed live, AES-256 confirmed on all sensitive fields, audit logging confirmed on CNIC/health access.
- Load-check the proximity query path specifically — this is our tightest KPI (<2s per NFR 1.2).
- Final KPI checklist review against PRD Section 7.4 — be honest with the group about which KPIs (like 99.5% uptime) are realistically measurable at this stage vs. aspirational for V1.

ABDUL HAYY KHAN — deployment

- Deploy develop → staging on Vercel, main → production only after Phase 4 sign-off and Phase 5 testing pass.
- Confirm environment variables/secrets are set in Vercel's dashboard (Supabase connection string, Firebase service account/config), never committed to the repo.
- Final walkthrough and sign-off with the whole team before we call V1 done.

A Few Things I Want Everyone to Keep in Mind

- This is our own product decision to go web instead of native — treat the wireframes as a reference for content and flow, not a pixel spec. Use your judgment to make it work well in a browser.
- If you're blocked because a shared model, schema, or another feature's endpoint isn't ready yet, say so early in the group instead of quietly working around it — that's usually how integration ends up painful.
- AI tools are here to make us faster, not to replace understanding your own code. If you can't explain what your PR does in your own words, don't merge it.
- Copilot's review is a second pair of eyes, not a rubber stamp — and it's not a substitute for a human review either.
- We're building something that people will actually reach for in a real emergency. That's worth the extra care around privacy, verification, and reliability that Section 7 of the PRD asks for — please don't treat NFRs as optional polish.

Questions, blockers, or disagreements with anything in this plan — bring them to the group chat early. Let's build this properly.

— Abdul Hayy Khan, Team Lead

https://github.com/abdulhayykhan/QATRA-Web-App/